Học viện Công nghệ Bưu chính Viễn thông
Khoa Công nghệ thông tin 1

Nhập môn trí tuệ nhân tạo

Giải quyết vấn đề bằng tìm kiếm

Ngô Xuân Bách

Nội dung

- Bài toán tìm kiếm trong không gian trạng thái
- Một số ví dụ
- Các thuật toán tìm kiếm cơ bản

Nội dung

- Bài toán tìm kiếm trong không gian trạng thái
  - Tìm kiếm và khoa học trí tuệ nhân tạo
  - Phát biểu bài toán tìm kiếm

- Các tiêu chuẩn đánh giá thuật toán tìm kiếm

- Một số ví dụ
- Các thuật toán tìm kiếm cơ bản

Tìm kiếm & khoa học trí tuệ nhân tạo

- Nhiều vấn đề (bài toán) có thể phát biểu và giải quyết dưới dạng tìm kiếm

- Trò chơi: tìm kiếm nước đi tối ưu (mang lại lợi thế)
- Lập thời khóa biểu: tìm kiếm phương án sắp xếp thỏa mãn yêu cầu đề ra (thỏa mãn ràng buộc)
- Tìm đường: tìm đường đi tối ưu (chiều dài, thời gian, giá, …)

- Tìm kiếm là một trong những hướng nghiên cứu quan trọng của trí tuệ nhân tạo
  - Phát triển các thuật toán tìm kiếm hiệu quả (đặc biệt trong những trường hợp không gian tìm kiếm có kích thước lớn)
  - Là cơ sở cho nhiều nhánh nghiên cứu khác của trí tuệ nhân tạo
    - Học máy, xử lý ngôn ngữ tự nhiên, suy diễn

Phát biểu bài toán tìm kiếm

Một bài toán tìm kiếm được phát biểu thông qua 5 thành phần sau

1. Tập hữu hạn các trạng thái có thể: ••

2. Tập các trạng thái xuất phát: $\mathcal{S} \subseteq \mathcal{S}$

3. Hành động hay hàm nối tiếp hay toán tử $\mathcal{A}(\mathcal{S})$, là tập các trạng thái nhận được từ trạng thái $\mathcal{S}$ do kết quả thực hiện hành động hay toán tử.

4. Xác định đích:
   - Tường minh, cho bởi tập đích $G \subseteq \mathcal{S}$
   - Không tường minh, cho bởi một số điều kiện

5. Giá thành đường đi
   - Ví dụ, tổng khoảng cách, số lượng hành động, …
   - $\mathcal{C}(\mathcal{S}, \mathcal{A}, \mathcal{S}) \geq 0$, là giá thành bước, từ trạng thái $\mathcal{S}$, thực hiện hành động $\mathcal{A}$, và chuyển sang trạng thái $\mathcal{S}$

Lời giải là chuỗi hành động cho phép di chuyển từ trạng thái xuất phát tới trạng thái đích

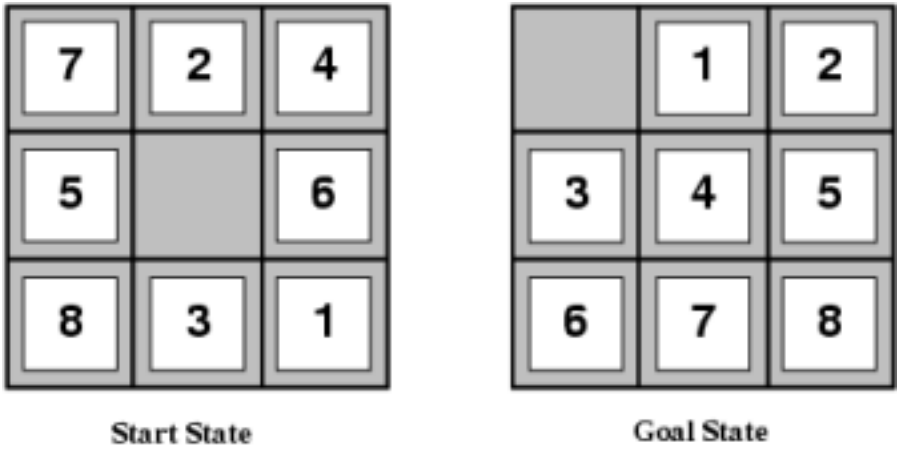
Các tiêu chuẩn đánh giá thuật toán tìm kiếm

- Độ phức tạp tính toán
  - Khối lượng tính toán cần thực hiện để tìm ra lời giải
  - Số lượng trạng thái cần xem xét trước khi tìm ra lời giải
- Yêu cầu bộ nhớ
  - Số lượng trạng thái cần lưu trữ đồng thời trong bộ nhớ khi thực hiện thuật toán
- Tính đầy đủ
  - Nếu bài toán có lời giải thì thuật toán có khả năng tìm ra lời giải đó không?
- Tính tối ưu
  - Nếu bài toán có nhiều lời giải thì thuật toán có cho phép tìm ra lời giải tốt nhất không?

Nội dung

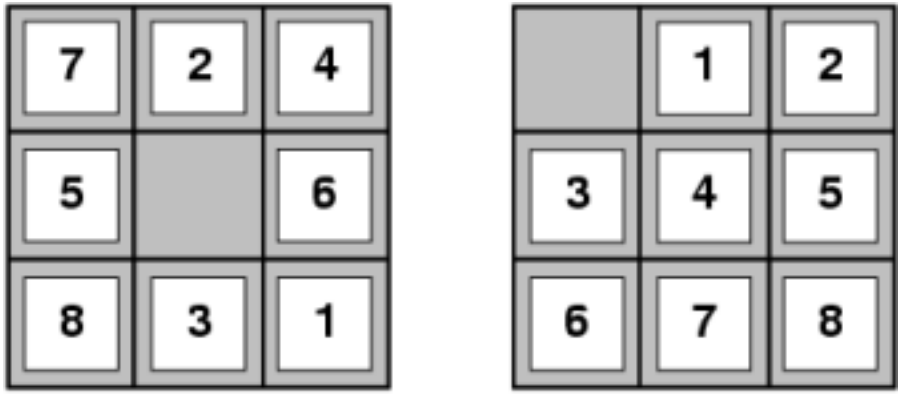
- Bài toán tìm kiếm trong không gian trạng thái
- Một số ví dụ
  - Trò chơi 8 ô
  - Bài toán 8 con hậu
- Các thuật toán tìm kiếm cơ bản

Trò chơi 8 ô (1/2)



(Russell & Norvig, 2010)

Trò chơi 8 ô (2/2)



Start State Goal State

(Russell & Norvig, 2010)

- Trạng thái: tổ hợp ví trí các ô
- Trạng thái xuất phát: một trạng thái bất kỳ
- Hành động: di chuyển ô trống trái, phải, lên, xuống
- Đích: so sánh với trạng thái đích (cho trước)
- Giá thành: số lần di chuyển

Bài toán 8 con hậu (1/2)

Đặt 8 con hậu lên bàn cờ vua 8x8 sao cho không có đôi hậu nào đe dọa nhau

Bài toán 8 con hậu (2/2)

Đặt 8 con hậu lên bàn cờ vua 8x8 sao cho không có đôi hậu nào đe dọa nhau

- Trạng thái: sắp xếp của 0 đến 8 con hậu trên bàn cờ
- Trạng thái xuất phát: không có con hậu nào trên bàn cờ
- Hành động: đặt một con hậu lên một ô trống trên bàn cờ
- Đích: 8 con hậu trên bàn cờ, không có 2 con nào đe dọa nhau

- Bài toán tìm kiếm trong không gian trạng thái
- Một số ví dụ
- Các thuật toán tìm kiếm cơ bản
  - Thuật toán tìm kiếm tổng quát
  - Tìm kiếm theo chiều rộng (Breadth-first search: BFS)
  - Tìm kiếm theo giá thành thống nhất (Uniform-cost search: UCS)
  - Tìm kiếm theo chiều sâu (Depth-first search: DFS)
  - Tìm kiếm sâu dần (Iterative deepening search: IDS)

Thuật toán tìm kiếm tổng quát (1/3)

- **Ý tưởng chung:** xem xét các trạng thái, sử dụng các hàm trạng thái để mở rộng các trạng thái đó cho tới khi đạt đến trạng thái mong muốn

- Mở rộng các trạng thái tạo ra "cây tìm kiếm"
  - Mỗi trạng thái là một nút
  - Các nút biên (nút mở) là nút đang chờ mở rộng tiếp
  - Nút đã mở rộng gọi là nút đóng

Thuật toán tìm kiếm tổng quát (2/3)

$$\text{\textbf{Đầu vào}: } h(\text{\textbf{Đầu ra}}, \text{\textbf{Khởi tạo}}, \text{\textbf{while}}, \text{\textbf{do}})$$

($\text{\textbf{Đầu ra}}$: không gian trạng thái, $\text{\textbf{Khởi tạo}}$: trạng thái bắt đầu, $\text{\textbf{while}}$: đích, $\text{\textbf{do}}$: hành động) **Đầu vào**: bài toán tìm kiếm

Đầu ra: trạng thái đích

Khởi tạo: $\text{\textbf{Đầu ra}} \leftarrow \text{\textbf{Khởi tạo}}$ ($\text{\textbf{Đầu ra}}$: danh sách các nút mở)

while($\text{\textbf{Đầu ra}} \neq \emptyset$) **do**

1. chọn nút $\text{\textbf{Khởi tạo}} \in \text{\textbf{Đầu ra}}$ và xóa $\text{\textbf{Khởi tạo}}$ khỏi $\text{\textbf{Đầu ra}}$
2. **if** $\text{\textbf{Khởi tạo}} \in \text{\textbf{while}}$, **return** (đường đi tới $\text{\textbf{Khởi tạo}}$)
3. thêm $\text{\textbf{Khởi tạo}}(\text{\textbf{Khởi tạo}})$ vào $\text{\textbf{Đầu ra}}$

return: Không tìm được lời giải

Thuật toán tìm kiếm tổng quát (3/3)

$$\text{\textbf{Đầu vào}: bài toán tìm kiếm}$$

(
: không gian trạng thái,
: trạng thái bắt đầu,
: đích,
: hành động) **Đầu vào**: bài toán tìm kiếm

Đầu ra: trạng thái đích

Khởi tạo: ← (: danh sách các nút mở)

Chọn nút n

while(≠ ∅) **do**

1. chọn nút ∈ và xóa khỏi 2. **if** ∈ , **return** (đường đi tới) 3. thêm () vào

return: Không tìm được lời giải

thế nào?

Ví dụ cây tìm kiếm

(Phuong TM, 2016)

Các chiến lược tìm kiếm

- Chiến lược tìm kiếm được xác định bởi thứ tự mở rộng các nút trên cây tìm kiếm

- Chiến lược tìm kiếm được đánh giá theo những tiêu chí sau:
  - đầy đủ: có tìm được lời giải không (nếu có)?
  - độ phức tạp tính toán: số lượng nút sinh ra
  - yêu cầu bộ nhớ: số lượng nút tối đa cần lưu trong bộ nhớ
  - tối ưu: có tìm được lời giải có giá thành nhỏ nhất không

Độ phức tạp được tính theo các tham số sau

- ⬚⬚: độ rẽ nhánh tối đa của cây tìm kiếm
  - ⬚⬚: độ sâu của lời giải
  - ⬚⬚: độ sâu tối đa của không gian trạng thái (có thể là ∞)

Tìm kiếm mù
(Tìm kiếm không có thông tin)

- Tìm kiếm mù (blind, uninformed) chỉ sử dụng các thông tin theo phát biểu của bài toán trong quá trình tím kiếm

- Các phương pháp tìm kiếm mù
  - Tìm kiếm theo chiều rộng (Breadth-first search: BFS)
  - Tìm kiếm theo giá thành thống nhất (Uniform-cost search: UCS)
  - Tìm kiếm theo chiều sâu (Depth-first search: DFS)
  - Tìm kiếm sâu dần (Iterative deepening search: IDS)

Tìm kiếm theo chiều rộng – BFS (1/4)

- **Nguyên tắc**: trong số những nút biên, lựa chọn nút nông nhất (gần gốc nhất) để mở rộng

Tìm kiếm theo chiều rộng – BFS (2/4)

- **Nguyên tắc**: trong số những nút biên, lựa chọn nút nông nhất (gần gốc nhất) để mở rộng

Tìm kiếm theo chiều rộng – BFS (3/4)

- **Nguyên tắc**: trong số những nút biên, lựa chọn nút nông nhất (gần gốc nhất) để mở rộng

Tìm kiếm theo chiều rộng – BFS (4/4)

Ghi nhớ đường đi

- Khi chuyển sang một nút cần ghi nhớ nút cha của nút đó bằng cách sử dụng con trỏ ngược
- Sau khi đạt tới đích, con trỏ ngược được sử dụng tìm đường đi trở lại nút xuất phát

Thuật toán BFS (1/2)

$$\text{BFS}(G, s, t, \text{move})$$

(G: không gian trạng thái, s: trạng thái bắt đầu, t: đích, move: hành

động) **Đầu vào**: bài toán tìm kiếm

Đầu ra: đường tới nút đích

Khởi tạo: tập các nút biên (nút mở) $\text{\diamond\diamond} = \text{\diamond\diamond}$

while($\text{\diamond\diamond} \neq \emptyset$) **do**

1. lấy nút đầu tiên $\text{\diamond\diamond}$ khỏi $\text{\diamond\diamond}$
2. **if** $\text{\diamond\diamond} \in \text{\diamond\diamond}$, **return** (đường đi tới $\text{\diamond\diamond}$)
3. thêm $\text{\diamond\diamond}(\text{\diamond\diamond})$ vào đuôi $\text{\diamond\diamond}$

return không tìm được đường đi

Thuật toán BFS (2/2)

$\text{\diamond\diamond\diamond\diamond\diamond\diamond}(\text{\diamond\diamond}, \text{\diamond\diamond}, \text{\diamond\diamond}, \text{\diamond\diamond})$

($\Phi\Phi$: không gian trạng thái, $\Phi\Phi$: trạng thái bắt đầu, $\Phi\Phi$: đích, $\Phi\Phi$: hành động) **Đầu vào**: bài toán tìm kiếm

Đầu ra: đường tới nút đích

Khởi tạo: tập các nút biên (nút mở) $\Phi\Phi = \Phi\Phi$

while($\Phi\Phi \neq \emptyset$) **do**

1. lấy nút đầu tiên $\Phi\Phi$ khỏi $\Phi\Phi$

2. **if** $\Phi\Phi \in \Phi\Phi$, **return** (đường đi tới $\Phi\Phi$) 3. thêm $\Phi\Phi(\Phi\Phi)$ vào đuôi $\Phi\Phi$

return không tìm được đường đi

Sử dụng cấu trúc hàng đợi FIFO

Tránh các nút lặp

- Có thể có nhiều đường đi cùng dẫn tới một nút
  - Thuật toán có thể mở rộng một nút nhiều lần
  - Có thể dẫn tới vòng lặp vô hạn

- Giải pháp
  - Không thêm nút vào hàng đợi nếu nút đã được duyệt hoặc đang nằm trong hàng đợi (chờ được duyệt)
    - Cần nhớ ít nút, thời gian kiểm tra nhanh
    - Tránh được vòng lặp vô hạn

Ví dụ BFS (1/2)

(Phuong TM, 2016)

Ví dụ BFS (2/2)

0		◆◆
1	◆◆	◆◆◆◆, ◆◆◆◆, ◆◆◆◆, ◆◆◆◆
2	◆◆◆◆	◆◆◆◆, ◆◆◆◆, ◆◆◆◆,◆◆◆◆
3	◆◆◆◆	◆◆◆◆, ◆◆◆◆,◆◆◆◆, ◆◆◆◆
4	◆◆◆◆	◆◆◆◆,◆◆◆◆, ◆◆◆◆, ◆◆◆◆
5	◆◆◆◆	◆◆◆◆, ◆◆◆◆, ◆◆◆◆, ◆◆◆◆
6	◆◆◆◆	◆◆◆◆, ◆◆◆◆, ◆◆◆◆
7	◆◆◆◆	◆◆◆◆, ◆◆◆◆
8	◆◆◆◆	◆◆◆◆

9	⬛⬛⬛⬛	Đích

Đường đi: ⬛⬛ ← ⬛⬛ ← ⬛⬛

Tính chất của BFS

- Đầy đủ?
  - Có (nếu ⬛⬛ là hữu hạn)
- Thời gian?
  - $1 + ⬛⬛ + ⬛⬛^{2} + ⬛⬛^{3} + \cdots + ⬛⬛^{⬛⬛} = ⬛⬛(⬛⬛^{⬛⬛})$
- Bộ nhớ?
  - $⬛⬛(⬛⬛^{⬛⬛})$ (lưu tất cả các nút)
- Tối ưu?
  - Có (nếu cost = 1 với mọi bước)
  - Do luôn tìm kiếm tất cả các nút ở mức trên trước khi tìm sang nút

ở mức dưới

- Bộ nhớ là vấn đề quan trọng hơn thời gian

Tìm kiếm theo giá thành thống nhất

- Trong trường hợp giá thành di chuyển giữa hai nút là không bằng nhau giữa các cặp nút
  - BFS không cho lời giải tối ưu
  - Cần sử dụng phương pháp tìm kiếm theo giá thành thống nhất (là một biến thể của BFS)

- **Phương pháp**: chọn nút có giá thành nhỏ nhất để mở rộng trước thay vì chọn nút nông nhất như trong BFS

Ví dụ UCS (1/2)

(Phuong TM, 2016)

Ví dụ UCS (2/2)

0		◆◆(0)
1	◆◆	◆◆$_{◆◆}$(55), ◆◆$_{◆◆}$(42), ◆◆$_{◆◆}$(48), ◆◆$_{◆◆}$(72)
2	◆◆$_{◆◆}$	◆◆$_{◆◆}$ 55 , ◆◆$_{◆◆}$ 48 , ◆◆$_{◆◆}$ 72 , ◆◆$_{◆◆}$(82)
3	◆◆$_{◆◆}$	◆◆$_{◆◆}$ 55 , ◆◆$_{◆◆}$ 72 , ◆◆$_{◆◆}$(82), ◆◆$_{◆◆}$(121)
4	◆◆$_{◆◆}$	◆◆$_{◆◆}$ 72 , ◆◆$_{◆◆}$(82), ◆◆$_{◆◆}$(121), ◆◆$_{◆◆}$(100)
5	◆◆$_{◆◆}$	◆◆$_{◆◆}$(82), ◆◆$_{◆◆}$(121), ◆◆$_{◆◆}$(100), ◆◆$_{◆◆}$(154)
6	◆◆$_{◆◆}$	◆◆$_{◆◆}$(121), ◆◆$_{◆◆}$(100), ◆◆$_{◆◆}$(132)
7	◆◆$_{◆◆}$	◆◆$_{◆◆}$(121), ◆◆$_{◆◆}$(132)

8	$\text{??}_{\text{??}}$	$\text{??}_{\text{??}}(132)$
9	$\text{??}_{\text{??}}$	Đích

Cập nhật đường đi tới ??

Đường đi: ?? ← ?? ← ?? ← ??

Tìm kiếm theo chiều sâu – DFS

(1/4)

 Nguyên tắc: trong số những nút biên, lựa chọn nút sâu nhất (xa gốc nhất) để mở rộng

Tìm kiếm theo chiều sâu – DFS (2/4)

- **Nguyên tắc**: trong số những nút biên, lựa chọn nút sâu nhất (xa gốc nhất) để mở rộng

Tìm kiếm theo chiều sâu – DFS (3/4)

- **Nguyên tắc**: trong số những nút biên, lựa chọn nút sâu nhất (xa gốc nhất) để mở rộng

Tìm kiếm theo chiều sâu – DFS (4/4)

- **Nguyên tắc**: trong số những nút biên, lựa chọn nút sâu nhất (xa gốc nhất) để mở rộng

Thuật toán DFS (1/2)

$\text{\textbf{\textbf{\textbf

Thuật toán DFS (2/2)

$\text{\diamond\diamond\diamond\diamond\diamond\diamond}(\text{\diamond\diamond}, \text{\diamond\diamond}, \text{\diamond\diamond}, \text{\diamond\diamond})$

(\diamond\diamond: không gian trạng thái, \diamond\diamond: trạng thái bắt đầu, \diamond\diamond: đích, \diamond\diamond: hành động) **Đầu vào**: bài toán tìm kiếm

Đầu ra: đường tới nút đích

Khởi tạo: tập các nút biên (nút mở) $\text{\diamond\diamond} = \text{\diamond\diamond}$

while($\text{\diamond\diamond} \neq \emptyset$) **do**

1. lấy nút đầu tiên \diamond\diamond khỏi \diamond\diamond
2. **if** $\text{\diamond\diamond} \in \text{\diamond\diamond}$, **return** (đường đi tới \diamond\diamond) 3. thêm \diamond\diamond(\diamond\diamond) vào đầu \diamond\diamond

return không tìm được đường đi

Sử dụng cấu trúc ngăn xếp LIFO

Ví dụ DFS (1/2)

(Phuong TM, 2016)

Ví dụ DFS (1/2)

0		►►
1	►►	►►$_{►►}$, ►►$_{►►}$, ►►$_{►►}$, ►►$_{►►}$
2	►►$_{►►}$	►►$_{►►}$, ►►$_{►►}$, ►►$_{►►}$, ►►$_{►►}$
3	►►$_{►►}$	►►$_{►►}$, ►►$_{►►}$, ►►$_{►►}$, ►►$_{►►}$
4	►►$_{►►}$	►►$_{►►}$, ►►$_{►►}$, ►►$_{►►}$, ►►$_{►►}$
5	►►$_{►►}$	Đích

Đường đi: ►► ← ►► ← ►► ← ►► ← ►► Độ sâu: 4

Tính chất của DFS

- Đầy đủ?
  - Không : trong trường hợp không gian trạng thái có độ sâu vô hạn
- Tối ưu?
  - Không
- Thời gian?
  - ►►(►►►►): rất lớn nếu ►► lớn hơn ►►
  - Nếu có nhiều lời giải thì có thể nhanh hơn tìm kiếm theo chiều rộng nhiều
- Bộ nhớ?
  - ►► ►►►► : tốt hơn nhiều so với tìm kiếm theo chiều rộng

Tìm kiếm sâu dần – IDS (1/3)

- **Phương pháp**: tìm theo DFS nhưng không bao giờ mở rộng các nút có độ sâu quá một giới hạn nào đó. Giới hạn độ sâu sẽ được tăng dần cho đến khi tìm được lời giải.

Tìm kiếm sâu dần – IDS (2/3)

Tìm kiếm sâu dần – IDS (3/3)

Thuật toán IDS

$\text{\textbf{\textbf{\textbf

Tính chất của IDS

- Đầy đủ?
  - Có
- Tối ưu?
  - Có: nếu có nhiều lời giải, có thể tìm ra lời giải gần gốc nhất
- Bộ nhớ?
  - $\bullet\bullet\ \bullet\bullet\bullet\bullet$: nhỏ
- Thời gian?
  - $\bullet\bullet + 1\ 1 + \bullet\bullet\bullet\bullet + \bullet\bullet - 1\ \bullet\bullet^{2} + \ldots + 2\bullet\bullet^{\bullet\bullet - 1} +$

$1\text{\diamond\diamond}^{\text{\diamond\diamond}} = \text{\diamond\diamond}(\text{\diamond\diamond}^{\text{\diamond\diamond}})$ Mang ưu điểm của cả BFS và

DFS

Ví dụ IDS

(Phuong TM, 2016)

Tóm tắt

Complete ?	Yes	Yes	No	Yes
Optimal?	Yes	Yes	No	Yes
Time	◆◆(◆◆$^{◆◆}$)	◆◆(◆◆$^{◆◆}$ ◌ɜ$^{*◆◆}$)	◆◆(◆◆$^{◆◆}$)	◆◆(◆◆$^{◆}$◆)
Space	◆◆(◆◆$^{◆◆}$)	◆◆(◆◆$^{◆◆}$ ◌ɜ$^{*◆◆}$)	◆◆(◆◆◆◆)	◆◆(◆◆◆◆)

- Nên chọn BFS nếu độ phân nhánh ◆◆ nhỏ
- Nên chọn DFS nếu biết trước độ sâu tối đa và có nhiều trạng thái đích
- Nên chọn IDS nếu cây tìm kiếm có độ sâu ◆◆ lớn

Khi nào đưa nút lặp vào danh sách?

- BFS
  - **Không**: Việc đưa nút lặp vào hàng đợi không làm thay đổi thứ tự các nút duyệt trong hàng đợi. Không làm thay đổi nghiệm bài toán. Ngoài ra còn bị rơi vào vòng lặp.
- UCS
  - Trong trường hợp nút lặp có giá thành (chi phí) tốt hơn, nó sẽ được đưa lại danh sách (nếu đã phát triển rồi) hoặc cập nhật thay nút cũ có giá thành kém hơn (nếu đang trong danh sách)
- DFS
  - **Có**: Việc đưa nút lặp vào ngăn xếp sẽ làm thay đổi thứ tự duyệt các nút trong ngăn xếp (thay đổi nhánh tìm kiếm), và thay đổi nghiệm của bài toán.
  - Tuy nhiên nếu đây là một nút đã được duyệt rồi thì sẽ **không** đưa vào ngăn xếp nữa.
- IDS:
  - **Có**: Để đảm bảo tính tối ưu của thuật toán

Tìm theo hai hướng (1/2)

- **Phương pháp**: tìm kiếm đồng thời bắt nguồn từ nút xuất phát và nút đích
  - Tồn tại hai cây tìm kiếm, một cây có gốc là nút xuất phát, một cây có gốc là nút đích
  - Tìm kiếm kết thúc khi có lá của cây này trùng với lá của cây kia
- Minh họa cây tìm kiếm

(Phuong TM, 2016)

Tìm theo hai hướng (2/2)

- Chú ý
  - Cần sử dụng tìm theo chiều rộng
    - Tìm theo chiều sâu có thể không ra lời giải nếu hai cây tìm kiếm phát triển theo hai nhánh không gặp nhau
  - Cần xây dựng các hàm
    - ►►(►►): tập các nút con của ►►
    - ►►(►►): tập các nút cha của ►►

- Tính chất
  - Việc kiểm tra nút lá này có trùng với nút lá kia đòi hỏi tương đối nhiều thời gian (►►$^{►►}$ nút của cây này với ►►$^{►►}$ nút của cây kia)
  - Độ phức tạp tính toán là ►►(►►$^{►►/2}$)
    - Số lượng nút cần mở rộng của hai cây giảm đáng kể